OPIPE-06: Cross-Scope Design Patterns

> **Series:** OPIPE | **Notebook:** 6 of 6 | **Created:** March 2026 | **Last Updated:** 04/04/2026

Correlating Logs, Spans, Metrics, and Events Across Scopes

OpenPipeline's six scopes process data independently — but observability value comes from **correlating** across them. A slow API response (span) may be caused by a database timeout (log) that triggers a Davis problem (event) and degrades an SLO metric. This notebook covers design patterns that connect the dots across scopes, cascade processing for derived data, and a production readiness checklist.

For individual scope deep-dives, see **OPIPE-02** (spans), **OPIPE-03** (metrics), **OPIPE-05** (events). For log processing, see **OPLOGS-01**. For trace analysis, see **SPANS-04: Service Dependencies & Flow Analysis**.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail
Permissions	`storage:logs:read`, `storage:spans:read`, `storage:metrics:read`, `storage:events:read`, `storage:bizevents:read`
Recommended	Complete **OPIPE-01 through OPIPE-05**

1. Shared Dimensions: The Correlation Key

Cross-scope correlation works when different data types share **common dimension values**. If your logs and spans both carry `service.name` and `k8s.namespace.name`, you can join them in DQL.

Correlation Fields Across Scopes

Field	Logs	Spans	Metrics	Events	Bizevents
`dt.entity.service`	Yes	Yes	Yes	Yes	—
`dt.entity.host`	Yes	—	Yes	Yes	—
`service.name`	Sometimes	Yes	—	—	—
`k8s.namespace.name`	Yes	Yes	Yes	—	—
`k8s.deployment.name`	Yes	Yes	Sometimes	—	—
`trace.id`	Sometimes	Yes	—	—	—
`dt.security_context`	If enriched	If enriched	If enriched	If enriched	If enriched

The Enrichment Strategy

If a critical dimension is missing from one scope, **add it via OpenPipeline enrichment** so that cross-scope queries work:

Scope	Missing Field	How to Add
Logs	`service.name`	Enrich from `dt.entity.service` via entity lookup
Spans	`k8s.deployment.name`	Enrich from `k8s.pod.name` by stripping replica suffix
Metrics	`k8s.namespace.name`	Enrich from entity tags or extension configuration

```
// Cross-scope: Correlate error logs with error spans by service entity
fetch logs, from:-1h
| filter loglevel == "ERROR" and isNotNull(dt.entity.service)
| summarize error_logs = count(), by:{dt.entity.service}
| lookup [
    fetch spans, from:-1h
    | filter http.response.status_code &gt;= 500 and
isNotNull(dt.entity.service)
    | summarize error_spans = count(), by:{dt.entity.service}
  ], sourceField:dt.entity.service, lookupField:dt.entity.service, fields:
{error_spans}
| sort error_logs desc
| limit 10
```

2. Pattern: Same Metric from Multiple Scopes

A powerful validation pattern: extract the **same metric** from two different data sources and compare them. If they diverge, one pipeline has a configuration problem.

Example: Error Count from Logs vs. Spans

Source	Metric Key	Extraction Rule
Logs	`log.error_count`	Count of `loglevel == "ERROR"`, by `dt.entity.service`
Spans	`span.error_count`	Count of `http.response.status_code >= 500`, by `dt.entity.service`

These metrics should track each other. If logs show 10x more errors than spans, it could mean:
- Application-level errors are logged but not reflected in HTTP status codes
- Span sampling is dropping error spans (see **OPIPE-03** on sampling-aware metrics)
- The log pipeline is catching errors from non-HTTP sources (background jobs, message consumers)

The comparison itself is diagnostic — the divergence tells you something about your data.

```
// Compare: Error counts from logs vs. spans by service
fetch logs, from:-1h
| filter loglevel == "ERROR" and isNotNull(dt.entity.service)
| summarize log_errors = count(), by:{dt.entity.service}
| lookup [
    fetch spans, from:-1h
    | filter http.response.status_code &gt;= 500 and
isNotNull(dt.entity.service)
    | summarize span_errors = count(), by:{dt.entity.service}
  ], sourceField:dt.entity.service, lookupField:dt.entity.service, fields:
{span_errors}
| fieldsAdd ratio = if(isNotNull(span_errors) and span_errors &gt; 0,
    then: round(toDouble(log_errors) / toDouble(span_errors), decimals: 1),
    else: -1.0)
| sort log_errors desc
| limit 10
```

3. Pattern: Cascade Processing

Cascade processing creates a chain of derived data across scopes:

```
Span → generates Event → Event triggers Metric → Metric drives SLO
```

Example: Slow Transaction Cascade

Stage	Scope	Configuration	Output
1. Detect	Spans	Filter: `span.kind == "server"` AND `duration > 5s`	Matching spans

Stage	Scope	Configuration	Output
2. Generate	Spans → Events	Event generation rule: create event with `event.type = "slow_transaction"`	Event record per slow span
3. Extract	Events → Metrics	Metric extraction: `slow_transaction.count` by `service.name`	Metric time series
4. Alert	Metrics → SLO	SLO: `slow_transaction.count < 10 per 5m` per service	SLO breach triggers alert

Design Considerations

- **Each stage adds latency** — The full cascade may take seconds from span ingestion to SLO evaluation
- **Failures propagate** — If the span pipeline drops the slow span, the event is never generated, and the metric never counts it
- **Test end-to-end** — After configuring a cascade, verify data flows through every stage

```
// Identify slow transactions that could trigger a cascade
fetch spans, from:-1h
| filter span.kind == "server" and duration &gt; 5000000000
| summarize slow_count = count(), avg_duration_ms = avg(toDouble(duration) /
1000000),
    by:{service.name}
| sort slow_count desc
| limit 10
```

4. Pattern: Unified Bucket Families

When related data across scopes should share the same lifecycle and access controls, use a **bucket family** naming convention:

Bucket Name	Scope	Contents	Retention
`checkout_logs`	Logs	Checkout service application logs	35 days
`checkout_spans`	Spans	Checkout service trace data	14 days

Bucket Name	Scope	Contents	Retention
`checkout_events`	Events	Checkout-related Davis events	35 days
`checkout_bizevents`	Bizevents	Purchase and cart business events	90 days

Benefits

- **Consistent naming** — `checkout_*` makes it obvious which buckets belong together
- **Unified access control** — IAM policies can use `dt.system.bucket` patterns with wildcards
- **Coordinated retention** — Related data expires in a logical sequence (spans first, then logs, business events last)

Querying Across a Bucket Family

```
// Query all buckets in a family to understand data distribution
fetch logs, from:-1h
| filter matchesValue(dt.system.bucket, "checkout_*")
| summarize log_count = count()
| append [
    fetch spans, from:-1h
    | filter matchesValue(dt.system.bucket, "checkout_*")
    | summarize span_count = count()
  ]
```

5. When NOT to Use OpenPipeline

OpenPipeline is powerful but not always the right tool. Prefer DQL query-time processing when:

Scenario	Why Not OpenPipeline	Better Approach
Exploratory analysis	You do not know what patterns you are looking for yet	DQL ad-hoc queries
Frequently changing logic	Pipeline changes require deployment; DQL changes are instant	DQL with dashboard variables

Scenario	Why Not OpenPipeline	Better Approach
Complex joins	OpenPipeline cannot join across scopes at ingestion	DQL `lookup` and `join` at query time
One-time investigations	Configuring a pipeline for a single investigation is overhead	DQL `parse` with DPL patterns
Historical data	OpenPipeline only processes new data — it cannot reprocess historical records	DQL for retroactive analysis
Correlation dashboards	Cross-scope correlation requires data from multiple tables	DQL with `append` and `lookup`

The Rule of Thumb

OpenPipeline for permanent, irreversible actions (drop, mask, route, extract). **DQL for everything else.**

If you are unsure whether a transformation belongs in OpenPipeline or DQL, ask: *"Will I regret not having the raw data?"* If yes, keep it raw and process at query time.

6. Production Readiness Checklist

Before deploying a new or modified OpenPipeline configuration to production:

Pipeline Design

- [ ] Each pipeline has a single, clear purpose (one source type per pipeline)
- [ ] Routing rules are ordered from most specific to least specific
- [ ] Default pipeline catches only unmatched data with short retention
- [ ] No overlapping routing conditions between pipelines

Processing Rules

- [ ] Masking rules applied BEFORE any other processing
- [ ] Drop rules filter noise before extraction (cost efficiency)
- [ ] Parsing rules tested against representative samples
- [ ] Enrichment fields have defined values for all conditions (no nulls)

Metric Extraction

- [ ] Cardinality estimated for each extracted metric (see **OPIPE-04**)
- [ ] No unique-per-record dimensions (`trace.id` , `span.id` , `content`)
- [ ] Sampling-aware extraction for span-derived count metrics (see **OPIPE-03**)
- [ ] Metric keys follow naming convention: `scope.metric_name` (e.g., `span.request_count`)

Security & Governance

- [ ] `dt.security_context` set on all scopes that need access control
- [ ] Security events routed to dedicated long-retention buckets
- [ ] Compliance-relevant data never dropped or sampled
- [ ] PII masking verified on all scopes that handle user data

Testing

- [ ] End-to-end data flow verified: ingestion → pipeline → bucket → query
- [ ] Cascade chains tested: span → event → metric → SLO (if applicable)
- [ ] Volume monitoring in place to detect unexpected spikes or drops
- [ ] Rollback plan documented (revert to previous pipeline configuration)

Summary

In this notebook you learned:

- **Shared dimensions** — Cross-scope correlation requires common fields like `dt.entity.service` and `k8s.namespace.name`
- **Same metric from multiple scopes** — Extract matching metrics from logs and spans to validate pipeline health
- **Cascade processing** — Chain span → event → metric → SLO for automated detection and alerting
- **Unified bucket families** — Name related buckets with a common prefix for consistent lifecycle management

- **When NOT to use OpenPipeline** — Keep data raw for exploratory analysis, changing logic, and one-time investigations
- **Production readiness** — Checklist covering pipeline design, processing rules, metric extraction, security, and testing

---

What's Next?

You have completed the OPIPE series. From here:

- **OPLOGS** — Deep-dive into log processing: **OPLOGS-01: OpenPipeline Fundamentals**
- **SPANS** — Master trace analysis: **SPANS-01: Fundamentals**
- **OPMIG** — Migrating from classic logs: **OPMIG-01: Why Migrate**
- **ORGNZ** — Data organization and governance: **ORGNZ-01: Introduction**

References

- OpenPipeline Documentation
- Grail Data Lakehouse
- DQL Cross-Data Queries
- SLO Configuration
